Arquitectura de Software

Laboratorio 1

Informe de Laboratorio

personapi-dotnet

Implementación de un Monolito con Patrón MVC y DAO

Estilo arquitectónico:	Monolito (MVC + DAO)
Plataforma:	.NET 10 LTS · ASP.NET Core MVC y Web API
Acceso a datos:	Entity Framework Core 10 (Database-First)
Persistencia:	SQL Server 2025 Express
Documentación de API:	OpenAPI 3 / Swagger (Swashbuckle)
Contenerización:	Docker / Docker Compose

Repositorio: `https://github.com/KatheBravo/personapi-dotnet`
TAG de entrega: `v1.2`
Fecha: octubre de 2026

Integrantes: Katherine Sofía Bravo Herrera · Daniel Avila Medina · Juan Sebastián Alvarez

Índice

1. Marco conceptual **3**

1.1. Arquitectura monolítica . . . 3
1.2. Patrón Modelo–Vista–Controlador (MVC) . . . 3
1.3. Patrón Data Access Object (DAO) / Repository . . . 3
1.4. Mapeo objeto-relacional (ORM) con Entity Framework Core . . . 3
1.5. API RESTful y especificación OpenAPI 3 / Swagger . . . 3
1.6. Contenedores y Docker Compose . . . 3
1.7. Stack tecnológico . . . 4

2. Diseño del sistema **4**

2.1. Modelo entidad–relación (base de datos) . . . 4
2.2. Arquitectura C4 . . . 5
2.2.1. Nivel 1: diagrama de contexto . . . 5
2.2.2. Nivel 2: diagrama de contenedores . . . 6
2.2.3. Nivel 3: diagrama de componentes . . . 7
2.2.4. Nivel 4: diagrama de código (patrón DAO) . . . 8
2.2.5. Diagrama de despliegue . . . 8
2.3. Diagramas de secuencia UML (operaciones CRUD) . . . 9
2.4. Catálogo de endpoints . . . 12

3. Procedimiento paso a paso **13**

3.1. Definición del esquema de base de datos . . . 13
3.2. Ingeniería inversa del modelo (Scaffold-DbContext) . . . 13
3.3. Implementación de la capa DAO . . . 13
3.4. Desarrollo de controladores . . . 14
3.5. Configuración de Swagger y OpenAPI 3 . . . 14
3.6. Contenerización y orquestación con Docker Compose . . . 14
3.7. Ejecución y verificación . . . 14

4. Conclusiones y lecciones aprendidas **14**

Índice de figuras

1. Modelo entidad–relación de personas, profesiones, estudios y teléfonos 5
2. Diagrama C4 de nivel 1: contexto del sistema 6
3. Diagrama C4 de nivel 2: contenedores 7
4. Diagrama C4 de nivel 3: componentes del contenedor web 7
5. Diagrama C4 de nivel 4: código del patrón DAO 8
6. Diagrama de despliegue en Docker 9
7. Diagrama de secuencia de la operación CREATE 10
8. Diagrama de secuencia de la operación READ 10
9. Diagrama de secuencia de la operación UPDATE 11
10. Diagrama de secuencia de la operación DELETE 12

Índice de tablas

1. Stack tecnológico del laboratorio 4
2. Operaciones CRUD y su comportamiento en la API 9
3. Catálogo de endpoints de la API REST 12
4. Puntos de acceso del entorno desplegado 14

1. Marco conceptual

1.1. Arquitectura monolítica

Una arquitectura monolítica agrupa todos los componentes del sistema (capa de presentación, lógica de negocio y capa de acceso a datos) dentro de una única unidad de despliegue y de código base. Entre sus ventajas se cuentan la simplicidad del desarrollo inicial, la facilidad para realizar pruebas integrales en local y la ausencia de latencia de red entre capas internas [1].

1.2. Patrón Modelo–Vista–Controlador (MVC)

El patrón MVC separa las responsabilidades de una aplicación interactiva en tres componentes [2]:

- **Modelo:** representa la estructura de datos, las reglas de validación y el estado de la información del dominio.
- **Vista:** es la representación gráfica con la que interactúa el usuario; en este proyecto son vistas `.cshtml` procesadas por el motor Razor.
- **Controlador:** intercepta las solicitudes HTTP entrantes, invoca los servicios del modelo y selecciona la vista adecuada para la respuesta.

1.3. Patrón Data Access Object (DAO) / Repository

El patrón DAO proporciona una interfaz abstracta que encapsula las operaciones de persistencia hacia el almacenamiento subyacente, en este caso una base de datos relacional. Al programar contra interfaces (`IPersonaRepository`, `IProfesionRepository`, `IEstudioRepository` e `ITelefonoRepository`), los controladores quedan desacoplados de la tecnología de acceso a datos (Entity Framework Core y SQL Server), lo que facilita la mantenibilidad y la sustitución tecnológica sin afectar las capas superiores [1].

1.4. Mapeo objeto-relacional (ORM) con Entity Framework Core

Entity Framework Core es el ORM oficial de Microsoft para .NET: permite mapear tablas de una base de datos relacional a clases POCO (*Plain Old CLR Objects*) de C# [3]. En este proyecto se empleó la técnica **Database-First** mediante el comando `Scaffold-DbContext`, que abstrae la ejecución de sentencias SQL mediante consultas LINQ fuertemente tipadas.

1.5. API RESTful y especificación OpenAPI 3 / Swagger

Una API RESTful modela los recursos del sistema bajo URI canónicas y usa los verbos del protocolo HTTP (`GET`, `POST`, `PUT`, `DELETE`) para reflejar las operaciones CRUD [5]. La especificación OpenAPI, implementada con Swashbuckle, genera una descripción formal en JSON y una consola interactiva (Swagger UI) que documenta esquemas, parámetros y códigos de estado HTTP.

1.6. Contenedores y Docker Compose

Docker empaqueta la aplicación junto con sus dependencias en imágenes reproducibles y portables. Docker Compose orquesta varios servicios [6]; aquí son el motor SQL Server 2025, un contenedor de inicialización que ejecuta los scripts DDL/DML y el contenedor de la aplicación ASP.NET Core, comunicados por una red interna.

1.7. Stack tecnológico

Tabla 1: Stack tecnológico del laboratorio

Componente	**Tecnología**	**Versión**
Lenguaje	C#	14
Framework	.NET (LTS) / ASP.NET Core MVC y Web API	10.0
Acceso a datos	Entity Framework Core (Database-First)	10.0
Base de datos	Microsoft SQL Server (edición Express)	2025
Documentación de API	OpenAPI 3.0 / Swashbuckle	10.2
Orquestación	Docker y Docker Compose	v2

2. Diseño del sistema

2.1. Modelo entidad–relación (base de datos)

El dominio del laboratorio se compone de cuatro entidades adaptadas desde el modelo relacional base [1]:

1. `persona`: individuos registrados (cédula como llave primaria, nombres, apellidos, género con restricción `CHECK` sobre `'M'`/`'F'` y edad).
2. `profesion`: catálogo de profesiones (identificador como llave primaria, nombre y descripción).
3. `estudios`: relación muchos a muchos entre personas y profesiones, con llave primaria compuesta `(id_prof, cc_per)`, fecha de graduación y universidad.
4. `telefono`: múltiples números asociados a una persona (llave foránea `duenio` con `ON DELETE CASCADE`).

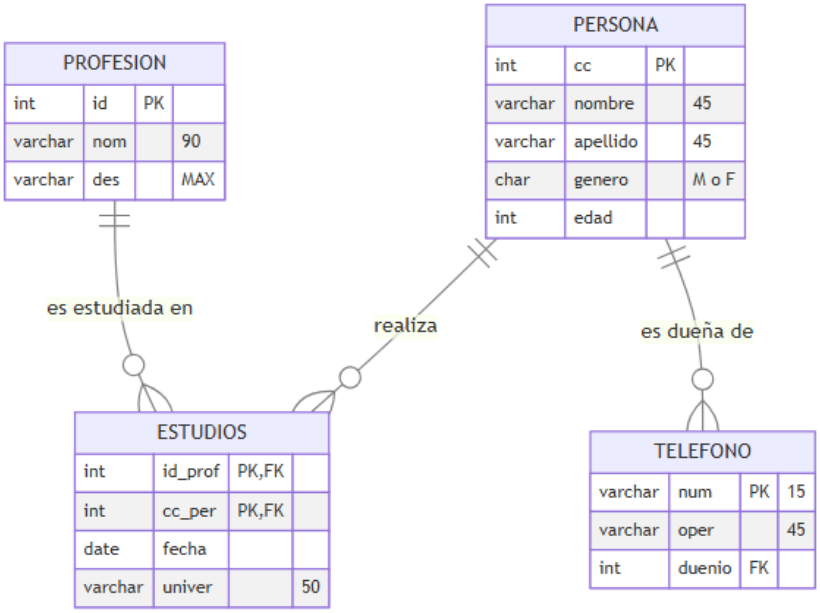


Figura 1: Modelo entidad–relación de personas, profesiones, estudios y teléfonos

La Figura 1 corresponde al esquema que crea `db/DDL.sql`. En `persona`, la cédula `cc` es la llave primaria, `genero` es un `CHAR(1)` con restricción `CK_persona_genero` y `edad` admite nulo con un rango permitido de 0 a 150. En `estudios`, las dos llaves foráneas (`estudio_persona_fk` y `estudio_profesion_fk`) tienen borrado en cascada, por lo que eliminar una persona o una profesión elimina también sus estudios. En `telefono`, el número es la llave primaria y `duenio` referencia a `persona` con el mismo comportamiento en cascada. El script es idempotente: usa `IF DB_ID(...) IS NULL` y `IF OBJECT_ID(...) IS NULL`, de modo que puede ejecutarse varias veces sin error.

2.2. Arquitectura C4

2.2.1. Nivel 1: diagrama de contexto

El sistema interactúa con dos perfiles de clientes: usuarios administrativos a través del navegador web (HTML) y desarrolladores o sistemas externos a través de la API RESTful (JSON) [4].

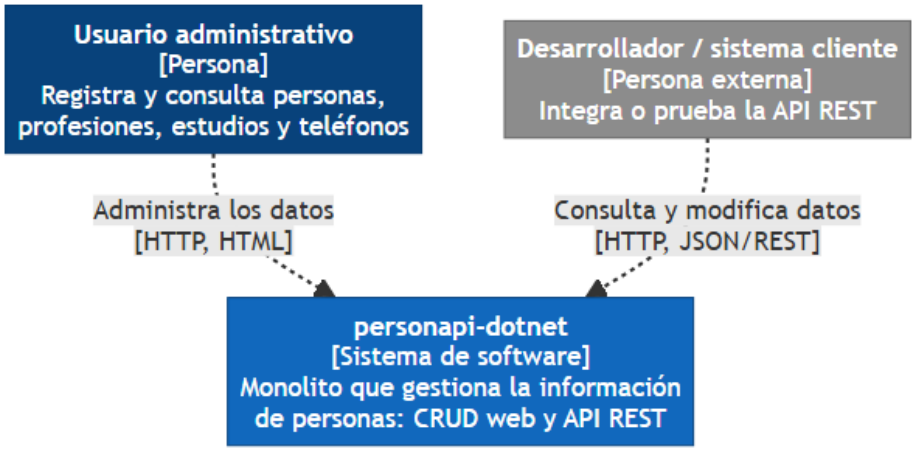


Figura 2: Diagrama C4 de nivel 1: contexto del sistema

2.2.2. Nivel 2: diagrama de contenedores

El sistema se divide en tres contenedores desplegables, definidos en `docker-compose.yml`:

- `personapi-app`: aplicación ASP.NET Core 10 que sirve las vistas Razor y los endpoints REST.
- `personapi-sqlserver`: instancia de Microsoft SQL Server 2025 Express.
- `personapi-db-init`: contenedor efímero con `sqlcmd` que aprovisiona el esquema (`DDL.sql`) y los datos semilla (`DML.sql`).

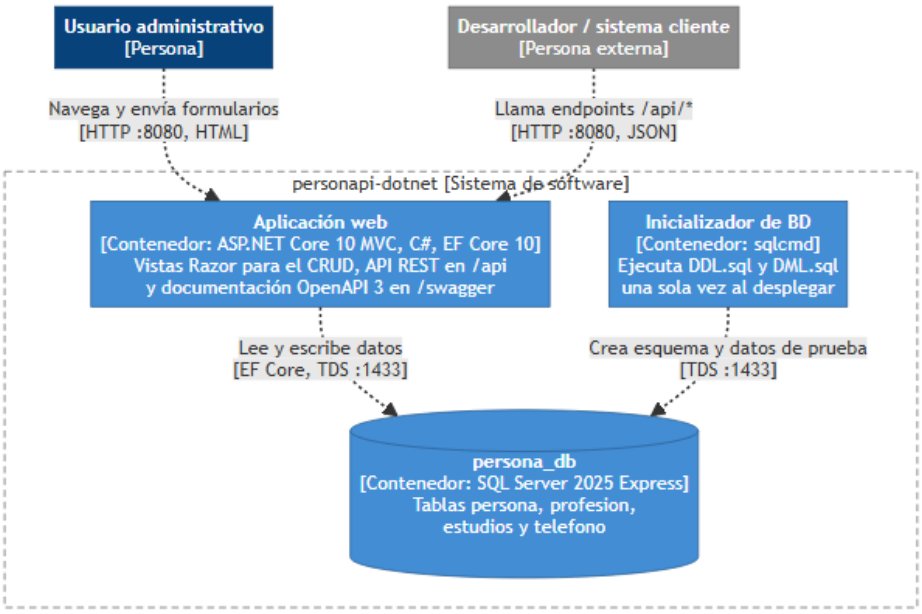


Figura 3: Diagrama C4 de nivel 2: contenedores

2.2.3. Nivel 3: diagrama de componentes

Dentro del contenedor web se estructuran los controladores MVC, los controladores de API REST, las clases DTO con sus mapeadores, la capa DAO (interfaces y repositorios) y el `PersonaDbContext`.

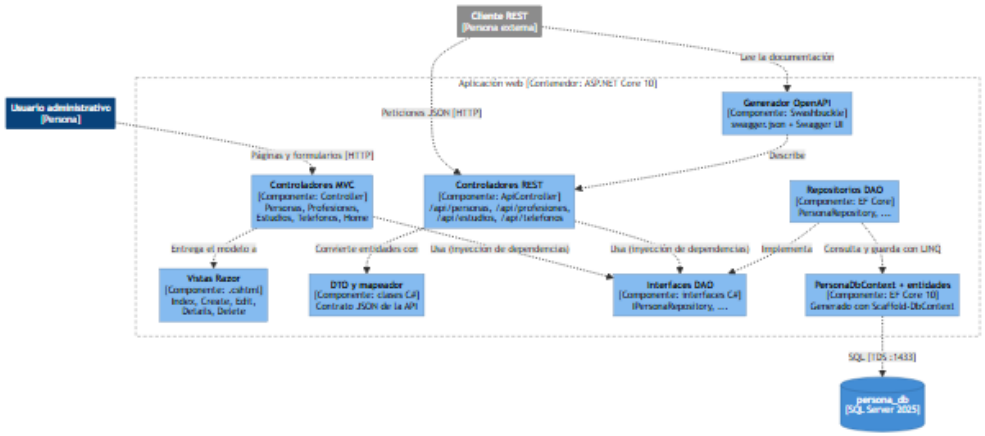


Figura 4: Diagrama C4 de nivel 3: componentes del contenedor web

La Figura 4 se corresponde con la estructura de carpetas del proyecto: `Controllers/` (vistas Razor) y `Controllers/Api/` (REST JSON) consumen los contratos de `Models/Interfaces/`, implementados en `Models/Repositories/` sobre el contexto definido en `Models/Entities/`; `Models/DTOs/` contiene los contratos de datos de la API.

2.2.4. Nivel 4: diagrama de código (patrón DAO)

Detalla la relación entre los controladores, la interfaz abstracta `IPersonaRepository`, su implementación `PersonaRepository` y el contexto de EF Core.

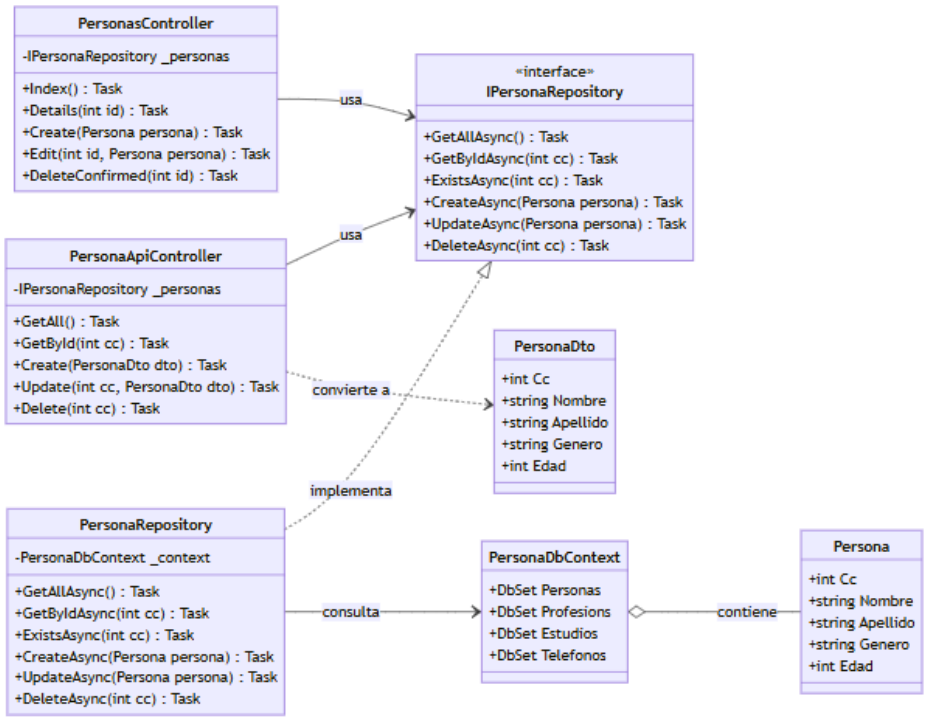


Figura 5: Diagrama C4 de nivel 4: código del patrón DAO

El desacoplamiento se materializa en `Program.cs`, donde cada contrato se registra con vida `Scoped` (por ejemplo, `AddScoped<IPersonaRepository, PersonaRepository>()`). Los controladores reciben la interfaz por el constructor y nunca instancian la clase concreta; `PersonaRepository` recibe a su vez el `PersonaDbContext`.

2.2.5. Diagrama de despliegue

Mapea los puertos hacia la máquina anfitriona (`8080` para web y API, `1433` para SQL Server), la red puente interna y el volumen de datos persistente `sqlserver-data`.

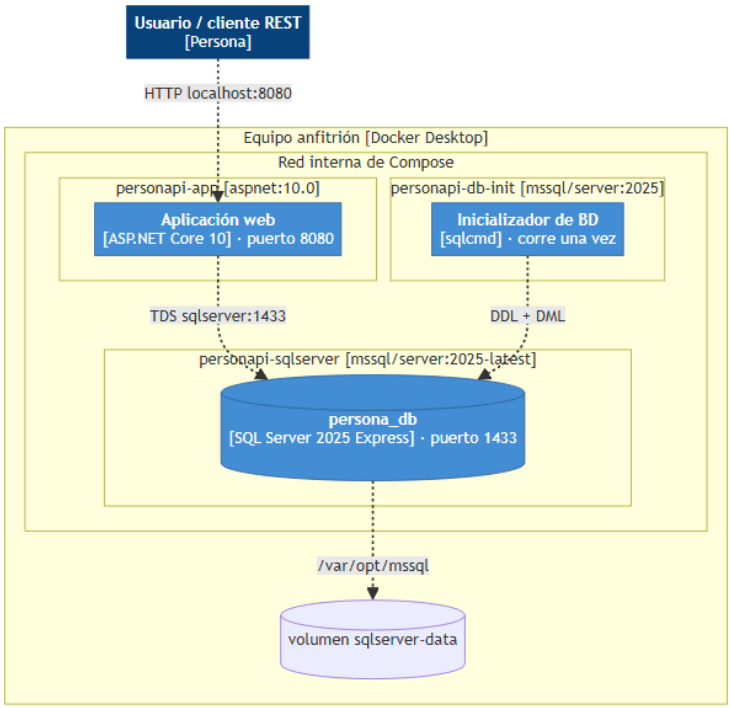


Figura 6: Diagrama de despliegue en Docker

El orden de arranque lo impone `depends_on`: `personapi-db-init` espera a que `personapi-sqlserver` supere su *healthcheck* (`service_healthy`), y `personapi-app` espera a que el inicializador termine correctamente (`service_completed_successfully`).

2.3. Diagramas de secuencia UML (operaciones CRUD)

Modelan la dinámica temporal entre los clientes, los controladores, los repositorios DAO, el contexto EF Core y la base de datos SQL Server.

Tabla 2: Operaciones CRUD y su comportamiento en la API

Operación	**Comportamiento**
CREATE	Valida el modelo con DataAnnotations, verifica la unicidad de la clave y persiste; responde `HTTP 201 Created`
READ	Ejecuta consultas con `AsNoTracking()` e `Include()` de relaciones; responde `HTTP 200 OK`
UPDATE	Valida la coherencia de la llave primaria y actualiza solo los campos mutables; responde `HTTP 204 No Content`
DELETE	Elimina el registro y activa el borrado en cascada (`ON DELETE CASCADE`) en SQL Server; responde `HTTP 204 No Content`

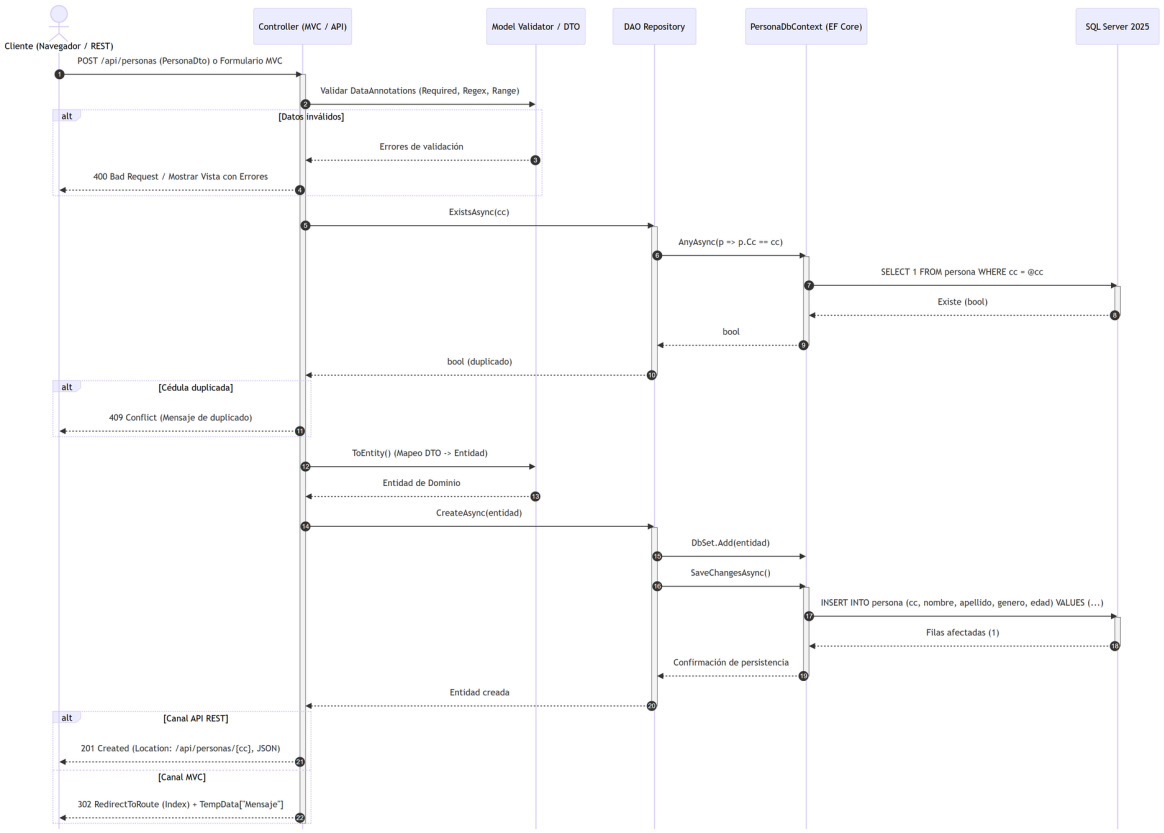


Figura 7: Diagrama de secuencia de la operación CREATE

La Figura 7 muestra el alta de un recurso. Si ya existe un registro con la misma clave, el controlador REST responde `409 Conflict` con un mensaje (por ejemplo, "Ya existe una persona con cédula ..."), y también cuando la persistencia falla por una violación de restricciones.

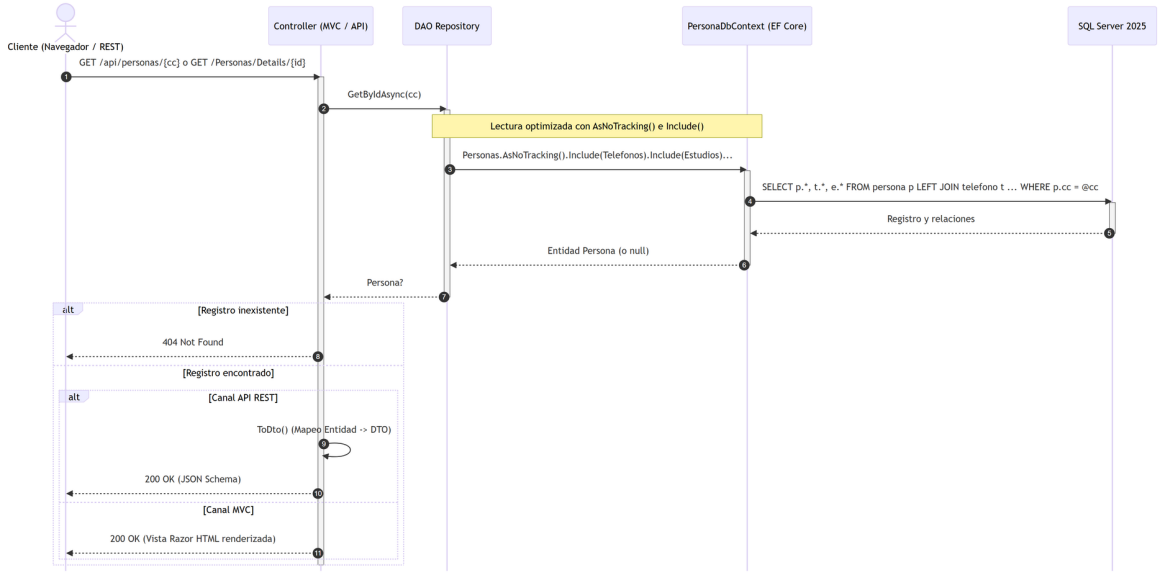


Figura 8: Diagrama de secuencia de la operación READ

La Figura 8 refleja la lectura. `PersonaRepository.GetByIdAsync` carga la persona con sus teléfonos y estudios (y la profesión de cada estudio) en una sola consulta, sin seguimiento de cambios, y el

controlador responde `404 Not Found` si no existe.

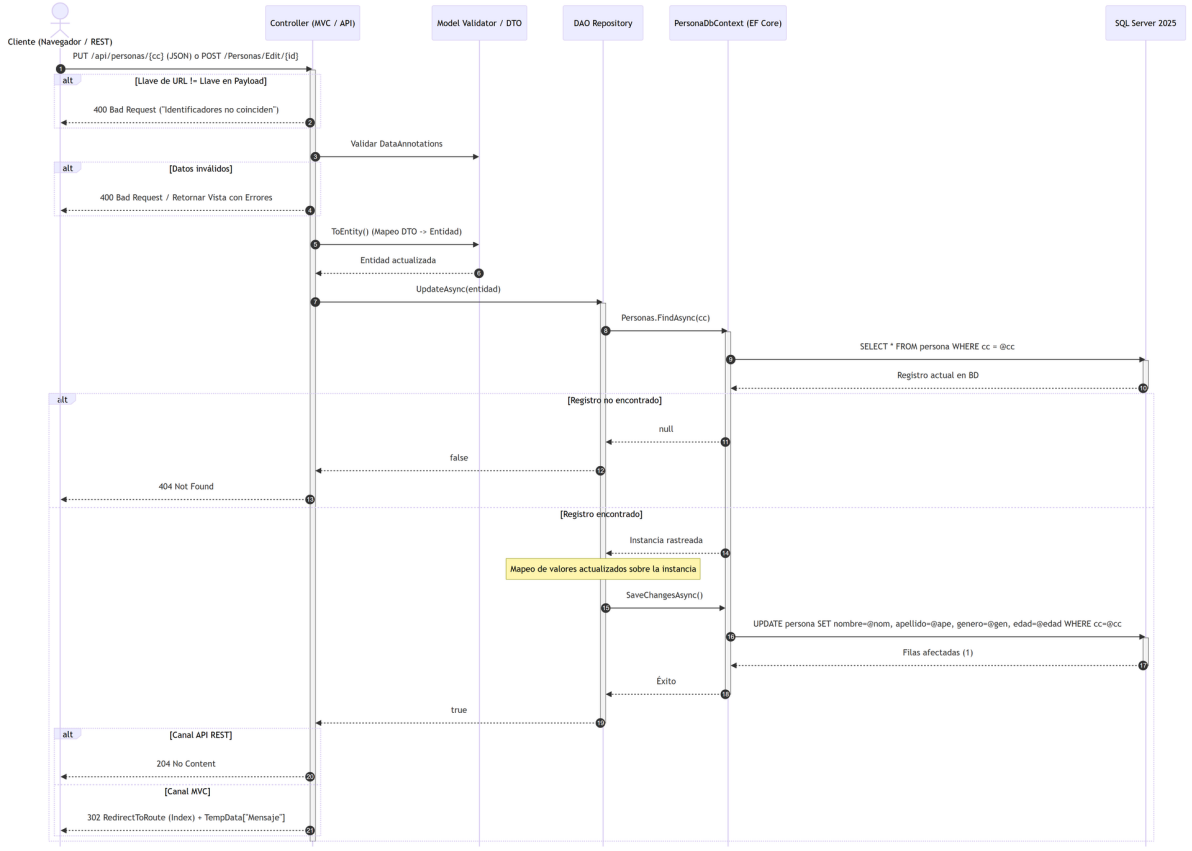


Figura 9: Diagrama de secuencia de la operación UPDATE

La Figura 9 muestra la actualización: el repositorio localiza la entidad existente y copia únicamente `Nombre`, `Apellido`, `Genero` y `Edad`, de modo que la cédula nunca se modifica.

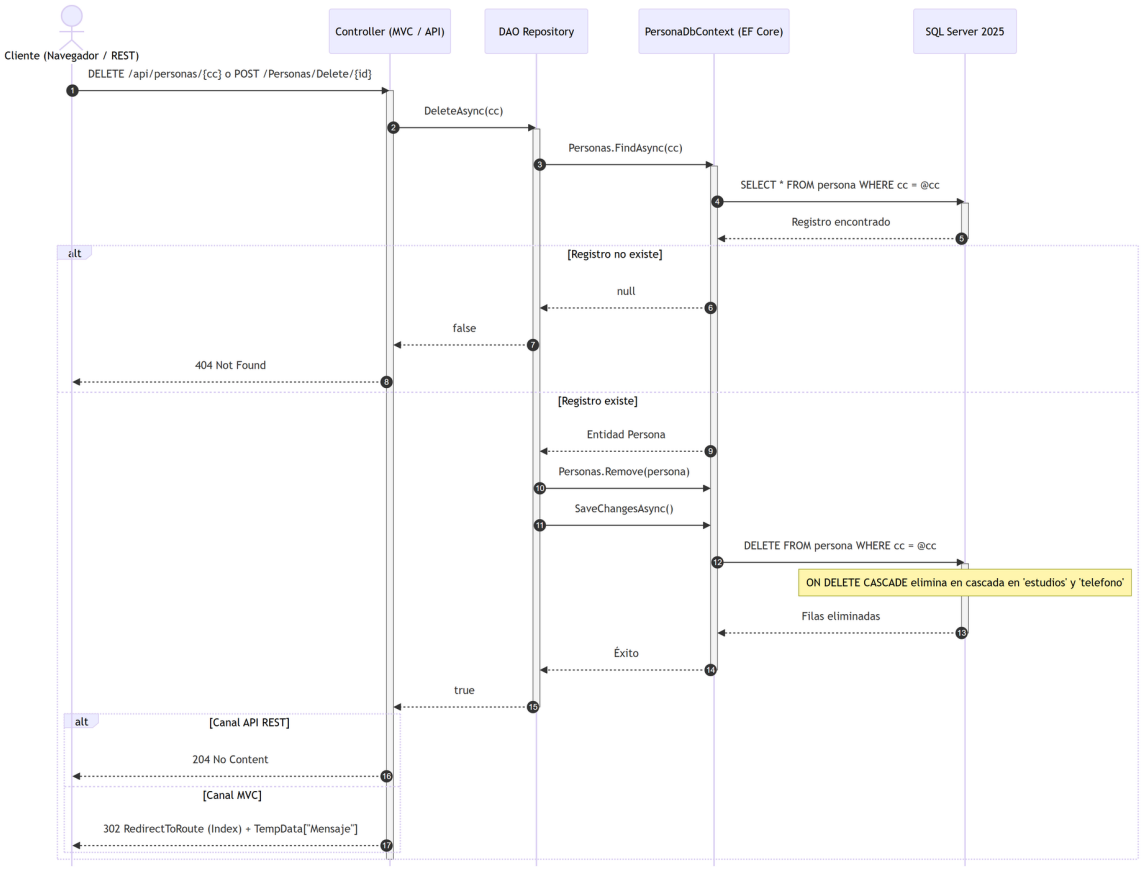


Figura 10: Diagrama de secuencia de la operación DELETE

La Figura 10 cubre la eliminación. El borrado en cascada lo resuelve SQL Server, no la aplicación: al borrar una persona desaparecen sus teléfonos y estudios.

2.4. Catálogo de endpoints

Los endpoints consumen y responden `application/json` con los códigos de estado `200`, `201`, `204`, `400`, `404` y `409` [1].

Tabla 3: Catálogo de endpoints de la API REST

Módulo	**Método**	**Endpoint**	**Acción**
Personas	`GET`	`/api/personas`	Lista general de personas
	`GET`	`/api/personas/{cc}`	Consulta por cédula
	`GET`	`/api/personas/{cc}/telefonos`	Teléfonos de la persona
	`GET`	`/api/personas/{cc}/estudios`	Estudios de la persona
	`POST`	`/api/personas`	Creación de persona
	`PUT`	`/api/personas/{cc}`	Actualización de persona
	`DELETE`	`/api/personas/{cc}`	Eliminación en cascada

Módulo	Método	Endpoint	Acción
Profesiones	GET	/api/profesiones	Lista de profesiones
	GET	/api/profesiones/{id}	Consulta por id
	POST	/api/profesiones	Creación de profesión
	PUT	/api/profesiones/{id}	Actualización de profesión
	DELETE	/api/profesiones/{id}	Eliminación en cascada
Estudios	GET	/api/estudios	Lista general de estudios
	GET	/api/estudios/{idProf}/{ccPer}	Consulta por clave compuesta
	POST	/api/estudios	Registro de estudio
	PUT	/api/estudios/{idProf}/{ccPer}	Actualización de fecha y universidad
	DELETE	/api/estudios/{idProf}/{ccPer}	Eliminación de estudio
Teléfonos	GET	/api/telefonos	Lista general de teléfonos
	GET	/api/telefonos/{num}	Consulta por número
	POST	/api/telefonos	Registro y asociación a un dueño
	PUT	/api/telefonos/{num}	Actualización de operador y dueño
	DELETE	/api/telefonos/{num}	Eliminación de teléfono

3. Procedimiento paso a paso

3.1. Definición del esquema de base de datos

- Se diseñó el script `db/DDL.sql` para Microsoft SQL Server, con idempotencia (`IF DB_ID(...) IS NULL`), asignando al usuario `sa` como propietario de la base y configurando integridad referencial con borrado en cascada (`ON DELETE CASCADE`).
- Se construyó el script `db/DML.sql` con datos representativos para las pruebas iniciales.

3.2. Ingeniería inversa del modelo (Scaffold-DbContext)

- A partir de la base de datos se ejecutó el *scaffold* de Entity Framework Core, que generó las entidades (`Persona`, `Profesion`, `Estudio`, `Telefono`) y el `PersonaDbContext`.
- Las validaciones se separaron en `EntityMetadata.cs` mediante `[ModelMetadataType]`, para proteger las reglas de negocio frente a futuros *scaffolds*.

3.3. Implementación de la capa DAO

- Se definieron contratos asíncronos en `Models/Interfaces/` (`IPersonaRepository`, etc.).
- Se crearon las implementaciones concretas en `Models/Repositories/`, con `AsNoTracking()` en las consultas de solo lectura e `Include()` para cargar entidades relacionadas.

3.4. Desarrollo de controladores

- **Controladores MVC** (`Controllers/*.cs`): implementan el flujo de vistas Razor con formularios interactivos, tokens antifalsificación (`[ValidateAntiForgeryToken]`) y mensajes de confirmación con `TempData`.
- **Controladores REST** (`Controllers/Api/*.cs`): implementan los endpoints JSON con DTOs, para evitar ciclos de serialización, y devuelven los códigos HTTP canónicos (`200`, `201`, `204`, `400`, `404`, `409`).

3.5. Configuración de Swagger y OpenAPI 3

En `Program.cs` se configuró Swashbuckle con metadatos descriptivos, etiquetas organizadas por entidad, inclusión de comentarios XML y opciones de Swagger UI (filtro de búsqueda, duración de las peticiones y lista expandida).

3.6. Contenerización y orquestación con Docker Compose

- Se creó un `Dockerfile` *multi-stage* que compila con `dotnet/sdk:10.0` y ejecuta con `dotnet/aspnet:10.0` bajo un usuario sin privilegios (`$APP_UID`).
- Se configuró `docker-compose.yml` para orquestar la base de datos con *healthchecks* y arranque coordinado (`depends_on`).

3.7. Ejecución y verificación

Todo el entorno está contenerizado, por lo que no se requiere instalar el SDK de .NET ni SQL Server en la máquina anfitriona.

Tabla 4: Puntos de acceso del entorno desplegado

Servicio	**URL / host**	**Detalle**
Interfaz web MVC	`http://localhost:8080`	Navegación y formularios Razor
Swagger UI	`http://localhost:8080/swagger`	Consola interactiva OpenAPI 3
Especificación OpenAPI	`http://localhost:8080/swagger/v1/swagger.json`	Esquema JSON de la API
SQL Server	`localhost:1433`	Acceso con el usuario `sa`

El ambiente se levanta con `docker compose up -d -build`. La suite `pruebas/prueba-humo.ps1` valida 89 aserciones sobre el CRUD de la API, las restricciones de unicidad, las llaves foráneas, el borrado en cascada y los flujos de las vistas MVC; el repositorio reporta el resultado `89 pruebas OK, 0 fallidas` [1].

4. Conclusiones y lecciones aprendidas

1. **Separación de responsabilidades.** El desacoplamiento entre controladores y persistencia mediante el patrón DAO permitió reutilizar la misma lógica de acceso a datos tanto en las vistas Razor como en los endpoints REST, evitando duplicidad de código.

2. **Uso de DTOs en APIs REST.** Exponer entidades de base de datos directamente en una API genera vulnerabilidades de asignación masiva (*over-posting*) y fallos de serialización circular por las propiedades de navegación de EF Core. Los DTOs y sus mapeadores resolvieron el problema de forma limpia.
3. **Reproducibilidad con Docker.** Estandarizar la solución en Docker Compose eliminó las diferencias de versiones entre entornos; por ejemplo, tener .NET 6 en el sistema anfitrión mientras la aplicación compila y corre en .NET 10 LTS dentro del contenedor.
4. **Documentación viva con OpenAPI.** Mantener Swagger sincronizado con los comentarios XML del código y los esquemas de validación de los DTOs asegura que la documentación técnica esté siempre actualizada sin esfuerzo adicional.

Referencias

[1] KatheBravo. *personapi-dotnet*: Laboratorio 1, monolito MVC + DAO con .NET, SQL Server, REST y Swagger. Repositorio, TAG `v1.2`. `https://github.com/KatheBravo/personapi-dotnet`

[2] Microsoft Docs. *ASP.NET Core MVC Overview*. `https://learn.microsoft.com/aspnet/core/mvc/overview`

[3] Microsoft Docs. *Entity Framework Core Database-First approach*. `https://learn.microsoft.com/ef/core/managing-schemas/scaffolding`

[4] S. Brown. *The C4 Model for Visualising Software Architecture*. `https://c4model.com/`

[5] R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures* (REST). Tesis doctoral, University of California, Irvine, 2000.

[6] Docker Documentation. *Docker Compose Overview*. `https://docs.docker.com/compose/`